

Reset detachment in the V1 GLIF model: investigation and implementation story

1. Why this investigation started

The original TensorFlow GLIF state update propagated gradients through the previous spike, `prev_z`, in both the voltage reset and the after-spike current (ASC) update:

```
new_asc = asc_decay * asc + expand_dims(prev_z, -1) * asc_amps
new_v = decay * v + current_factor * c1 - prev_z
```

The CUDA state-transition kernel was first aligned with this fully differentiable update. That restored the reset and ASC gradients through `prev_z` and removed the backward-only dampening of the voltage self-loop.

This version was correct relative to the active TensorFlow equations, but a full-model comparison showed a small performance cost: approximately 140 MiB more TensorFlow GPU memory and about 2.5% more runtime per steady-state step. Part of that difference came from a custom-gradient shape-safety expression that unnecessarily evaluated `zeros_like(output) + gradient` even when the gradient already had the correct shape. The wrapper was corrected to return compatible gradients directly and broadcast only scalar or genuinely incompatible cotangents.

That optimization addressed unnecessary TensorFlow operations, but it did not answer the scientific question: should the voltage reset be differentiable at all?

2. The reset-gradient question

For a simplified subtractive-reset neuron,

$$v_{t+1} = \alpha v_t + I_t - z_t, \quad z_t = H(v_t - \vartheta),$$

surrogate-gradient training replaces the derivative of the spike function with a smooth approximation, denoted by g_t .

If the reset is differentiable, the direct temporal voltage Jacobian is approximately

$$\frac{\partial v_{t+1}}{\partial v_t} = \alpha - g_t.$$

If the reset is detached, it becomes

$$\frac{\partial v_{t+1}}{\partial v_t} = \alpha.$$

This explains the initially surprising observation that detaching the reset can make gradients larger. The differentiable reset contributes the negative term $-g_t$, which can cancel part of the positive membrane-memory derivative. Detachment removes that cancellation. In an explicitly recurrent network, the complete Jacobian also contains recurrent synaptic, PSC, and ASC pathways, so a membrane decay below one does not by itself guarantee stable gradients.

3. What the literature supports

The supplied paper, Eshraghian et al., *Training Spiking Neural Networks Using Lessons From Deep Learning* (2023), recommends detaching the reset as a practical surrogate- gradient convention. Its main empirical basis is Zenke and Vogels (2021).

The more precise conclusion from the literature is:

- Reset detachment is a common and defensible default.
- The adverse behavior of differentiable resets is strongly coupled to surrogate derivative scale.
- Explicitly recurrent SNNs are included in the empirical evidence; this is not only a result for layered feedforward SNNs.
- Reset detachment is not guaranteed to improve every task or every optimizer setup.
- The literature generally stabilizes training through surrogate normalization, recurrent-path control, initialization, and gradient clipping. It does not prescribe constant backward-only dampening of the passive voltage self-loop.

The full source review is in [literature_review.md](#).

4. Why voltage-path dampening was reconsidered

The earlier experimental update used

```
dampened_v = straight_through_dampen(v, voltage_gradient_dampening)
new_v = decay * dampened_v + current_factor * c1 - stop_gradient(prev_z)
```

The forward value of `dampened_v` equals `v`, but its backward derivative is multiplied by `1 - voltage_gradient_dampening`. With a dampening value of `0.5`, the direct voltage derivative is therefore

$$0.5\alpha.$$

For the inspected network, `decay` was approximately 0.83--0.98, with a median near 0.93. The dampened backward multiplier was consequently only about 0.42--0.49. This is not mild stabilization: it strongly contracts membrane-based temporal credit on every timestep, including timesteps with no spike.

Voltage dampening may still be a useful model-specific ablation, but it should not be silently coupled to reset detachment. The final implementation therefore preserves the full voltage self-loop.

5. The final gradient design

The active TensorFlow membrane update is now

```
new_v = decay * v + current_factor * c1 - stop_gradient(prev_z)
```

Its intended gradient behavior is:

Path	Gradient behavior
<code>v -> new_v</code>	Full derivative <code>decay</code>
<code>c1 -> new_v</code>	Full derivative <code>current_factor</code>
<code>prev_z -> new_v</code>	Detached
<code>prev_z -> new_asc</code>	Differentiable through both ASC amplitudes
refractory state	Detached
hard-reset refractory voltage	Zero voltage gradient while clamped

Detaching only the voltage reset is important. Detaching `prev_z` from the ASC update would remove a separate spike-to-adaptation learning pathway and would be a different scientific intervention.

The differentiable CUDA kernel implements the same rule. Its `prev_z` gradient contains the two ASC-amplitude contributions but no longer contains the `-new_v_gradient` reset contribution. Forward CUDA equations are numerically unchanged.

6. Surrogate and recurrent-gradient settings

The project has three distinct controls that should not be conflated:

1. `dampening_factor` controls the amplitude of the spike surrogate derivative.
2. `recurrent_dampening_factor` scales gradients returned through recurrent synaptic current computation.
3. `voltage_gradient_dampening` was intended to attenuate the passive voltage self-loop.

The wrapper launcher previously defaulted `dampening_factor` to `1`, even though a nearby comment indicated `0.1`. It now defaults to `0.1`, matching `multi_training.py`. The recurrent dampening default remains `0.1`. The voltage-dampening compatibility flag now defaults to `0.0` and does not affect the active state equation.

These defaults implement the literature-aligned starting point: detached reset, smaller surrogate amplitude, recurrent-path dampening, and intact membrane credit assignment.

7. Global-gradient clipping

An optional `--global_clipnorm` training flag was added. Clipping is applied after mixed-precision loss-scale gradients have been unscaled and before `optimizer.apply_gradients`:

```
scaled loss
-> raw scaled gradients
-> mixed-precision unscaling
-> pre-clip global-norm measurement
-> optional global-norm clipping
-> optimizer update
```

This ordering is necessary because clipping loss-scaled gradients would make the effective threshold depend on the dynamic loss scale.

`None` gradients retain their positions so they remain aligned with the corresponding variables. When `--debug_gradients` is enabled, training prints the pre-clipping global norm. A value of `--global_clipnorm 0` disables clipping while retaining norm measurement.

Clipping is disabled by default. A threshold learned from a small network should not be assumed safe or useful for the 203k-neuron network.

8. Verification performed

Focused behavioral tests

A new state-transition test independently observes the two `prev_z` paths:

- the gradient from `new_v` to `prev_z` must be absent or zero;
- the gradient from `new_asc` to `prev_z` must equal the sum of the two ASC amplitudes.

The same behavior is tested through the TensorFlow and CUDA state-transition backends. Existing CUDA parity coverage also checks FP32, FP16, soft reset, and hard reset.

Gradient-clipping tests verify:

- correct global-norm scaling;
- preservation of missing gradients;
- reporting of the raw norm;
- no gradient modification when clipping is disabled.

The isolated-GPU CUDA suite completed with `13 passed`. Python compilation, Ruff lint, and `git diff --check` also passed.

Real 10k-neuron smoke test

One FP16 training update was run on the 10k-neuron network with:

- sequence length 100;
- segmented exact BPTT with chunk size 25;
- `dampening_factor=0.1`;
- `recurrent_dampening_factor=0.1`;
- no voltage dampening;
- clipping disabled for the baseline measurement.

Both sequential stimulus updates had finite gradients:

Branch	Pre-clip global norm	Maximum absolute gradient
Evoked	0.984	0.125
Spontaneous	0.852	0.081

The measured training update took about 18.3 seconds, including first-step graph work, and TensorFlow peak GPU memory was approximately 352 MiB for this small configuration.

The same seed was then run with `--global_clipnorm 0.9`. The evoked branch was clipped, reducing its maximum absolute gradient from approximately 0.125 to 0.115, while the spontaneous branch remained unchanged because its global norm was below 0.9. This verified the real training integration. It does **not** establish 0.9 as the recommended full-network threshold.

9. How to approach the next full-network run

The next step should be a short diagnostic pilot, not an immediate long training run. Use the intended full network and batch configuration with:

```
--dampening_factor 0.1
--recurrent_dampening_factor 0.1
--voltage_gradient_dampening 0.0
--global_clipnorm 0.0
--debug_gradients
```

Record pre-clip global norms for enough representative evoked and spontaneous updates to observe their typical range and upper tail. Then choose a clipping threshold that primarily catches exceptional updates rather than clipping nearly every step.

For example, a useful threshold can be based on a high percentile of stable baseline norms. The exact percentile is an experimental choice; the important diagnostic is the fraction of updates clipped. If almost every update clips, the threshold is masking an underlying scale problem rather than acting as a safety rail.

After selecting a threshold, compare at least these conditions using identical seeds and initial weights:

1. attached reset, surrogate amplitude 0.1, no voltage dampening;
2. detached reset, surrogate amplitude 0.1, no voltage dampening;
3. detached reset with the selected global clipping threshold;
4. detached reset with a different surrogate amplitude, such as 0.2 or 0.3;
5. only if instability remains, detached reset with mild voltage dampening as an explicitly separate ablation.

Judge performance by validation loss and biological activity metrics as well as training loss. Faster loss reduction is not useful if it results from pathological firing rates, collapsed activity, persistent clipping, or the loss of long-timescale credit assignment.

10. Current conclusion

The implementation now follows a coherent separation of concerns:

- voltage reset detachment defines how the discontinuous reset is treated;
- the surrogate amplitude controls the spike derivative;
- recurrent dampening controls explicit recurrent-current credit;
- the membrane self-loop retains its physical decay derivative;
- global clipping is an optional optimizer safety mechanism;

- ASC spike credit remains differentiable.

The small-network evidence shows that this combination compiles, matches across TensorFlow and CUDA, produces finite gradients, and supports clipping at the correct point in the mixed-precision training path. Whether it improves convergence on the full V1 model remains an empirical question requiring matched multi-seed experiments.

Primary literature

- Eshraghian et al. (2023), [Training Spiking Neural Networks Using Lessons From Deep Learning](#).
- Zenke and Vogels (2021), [The Remarkable Robustness of Surrogate Gradient Learning for Instilling Complex Function in Spiking Neural Networks](#).
- Gygax and Zenke (2025), [Elucidating the Theoretical Underpinnings of Surrogate Gradient Learning in Spiking Neural Networks](#).